Operating Systems | Assignment 10

xv6 x86 memory management, translation, page protection and replacement
Implementation and measured observations | 3 November 2026

Base and files

Built against MIT xv6-public (32-bit x86), commit eeb7b415dbcb12cc362d0783e41c3d1f44066b17. The assignment names walkpgdir() and usys.S, so this is the x86 base, not xv6-riscv. Modified files: kalloc.c, vm.c, sysproc.c, syscall.c, syscall.h, defs.h, user.h, usys.S and Makefile. Added programs: memtest.c, vttest.c, pagetest.c, pagereplace.c and supplemental vmcheck.c. Assignment changes are marked in source comments.

Q1. Physical page accounting

kalloc() increments the allocation count only when it successfully removes a page. kfree() counts runtime returns. Initial freerange() boot seeding is tracked separately and is not treated as runtime deallocation; otherwise allocated minus freed would be negative after boot. A snapshot is taken under kmem.lock. memstats() prints cumulative allocations, cumulative runtime frees, their difference (currently allocated pages), and a free-list count. These count allocator-managed pages, not reserved kernel memory or every page of installed RAM.

Measured before / after sbrk(16384) / after sbrk(-16384): allocated 490 / 494 / 494; freed 214 / 214 / 218; current 276 / 280 / 276; available 56789 / 56785 / 56789. The four-page allocation and release return current usage and availability exactly to baseline. Counts can vary with boot and process activity.

Q2. Virtual-to-physical translation

vtop(int va) reads its integer argument, rejects kernel or out-of-process addresses, and calls a vm.c helper using walkpgdir(..., alloc=0). A present, user-accessible PTE is required. The result is PTE_ADDR(*pte) | (va & 0xFFF), returned as int; invalid mappings return -1. No page-table entry is created by inspection.

vttest allocated three pages. In this run VA 0x3000 mapped to PA 0x0DEE3000, VA 0x4000 to 0x0DEE1000 and VA 0x5000 to 0x0DF22000. VA 0x307B mapped to 0x0DEE307B, preserving offset 0x7B. VA 0x67000 was rejected. Physical frames need not be adjacent.

Q3. Page permissions and fault demonstration

pageinfo() prints the physical frame number and P, W and U bits. setreadonly() clears PTE_W on a valid user page and reloads CR3 to invalidate cached TLB permissions. pagetest initializes a page to 42, displays W=1 then W=0, forks a child and attempts a volatile write. The child re-applies protection to exercise immediate invalidation. A pipe detects an unexpected successful write rather than assuming the child faulted.

Observed trap 14, error 7 at VA 0x3000: a present-page, user-mode write protection fault. xv6 kills the child only. The parent confirms that the protected write did not complete and can still read 42. Existing copyuvm() preserves PTE permissions during fork; no change to that routine is needed.

Replacement results and verification

Q4. FIFO, LRU and Optimal

pagereplace.c is a user-space simulation, not kernel swapping. Each run starts with empty frames and uses the required 20-reference string: 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1. It prints frames and HIT/FAULT after every reference for the three-frame runs, followed by results for three, four and five frames.

FIFO replaces the oldest inserted resident page; hits do not reset its age. LRU replaces the least recently used resident page and updates its age on hits. Optimal scans future references and replaces the page whose next use is farthest away, including pages never used again. Ties are broken by first frame position. Hit ratio = hits / 20; fault ratio = faults / 20. Ratios are printed as percentages with two decimals using integer arithmetic.

Frames	Algorithm	Faults	Hits	Hit ratio	Fault ratio
3	FIFO	15	5	25.00%	75.00%
3	LRU	12	8	40.00%	60.00%
3	Optimal	9	11	55.00%	45.00%
4	FIFO	10	10	50.00%	50.00%
4	LRU	8	12	60.00%	40.00%
4	Optimal	8	12	60.00%	40.00%
5	FIFO	9	11	55.00%	45.00%
5	LRU	7	13	65.00%	35.00%
5	Optimal	7	13	65.00%	35.00%

Optimal is best or tied at each capacity: 9 / 8 / 7 faults. LRU ties Optimal at four and five frames. FIFO has more faults at every tested capacity. The best observed count is seven faults, shared by LRU and Optimal with five frames. Optimal requires future knowledge, so it is a comparison bound rather than an online policy.

Verification and reproduction

Build: make xv6.img fs.img. Run: make qemu-nox CPUS=1, then memtest, vttest, pagetest, pagereplace, vmcheck and usertests. Tested using GCC 11.4.0 and QEMU 6.2.0, one virtual CPU and 512 MB machine RAM; xv6 PHYSTOP remains 224 MB. The full built-in usertests run reached ALL TESTS PASSED. Expected usertests protection traps and out-of-memory messages are intentional negative tests, not kernel panics.

vmcheck verifies page offsets and rejection of negative, kernel, heap-end and freed-page addresses. A separate Python implementation independently matched all nine replacement table rows. Evidence contains real QEMU serial output and terminal screenshots, with complete logs retained.